A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Arpita Sandesh Borulkar (63)

Under the Guidance of

Prof.T.H.Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]



BHARATI VIDYAPEETH

COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the **mini-project** entitled "**Implementation of Job Scheduling Using Graphs**" has been successfully completed by [**Arpita Sandesh Borulkar**], Roll Number [**63**], of the Department of Computer Engineering, **Bharati Vidyapeeth's College of Engineering**, **Lavale**, Pune,

Prof. T.H.Patil
(Guide)

Dr. Lalit.V.Patil
Head of Computer Engg.
Department

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. T.H.Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit.V.Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Student Name : Arpita Sandesh Borulkar

ABSTRACT

Job scheduling is an important process in computer systems in which different jobs are arranged and executed in a suitable order. Some jobs depend on the completion of other jobs, so it is necessary to determine the correct execution sequence.

This mini project, ****Implementation of Job Scheduling Using Graphs**,** demonstrates how graph data structures can be used to represent jobs and their dependencies. In this project, each job is represented as a vertex, and a directed edge represents a dependency between two jobs.

The project uses a Directed Acyclic Graph (DAG) to represent job dependencies and applies topological sorting to determine a valid order of execution. The program is implemented using Python and displays the scheduled order of jobs.

This project helps in understanding the practical applications of graphs, directed edges and topological sorting in task scheduling and dependency management.

Acknowledgement i
Abstract ii
Content iii
List Of Figures iv

TABLE OF CONTENTS

SR.NO	CONTENT	PAGE NO.
1	Introduction a) Overview b) Problem Statement c) Objectives	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirements	4
5	System Design a) Existing system b) Proposed system c) Graph representation	5
6	Result	6
7	Application	9
8	Future Scope	10

9 Conclusion 11

10 References 12

LIST OF FIGURES

Figure	Description	Page No.
Fig 1	UML Case Diagram	3
	System Design	5
Fig 5.1	System Code	7
Fig 5.2	Result	8

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview:

Job scheduling is the process of arranging different jobs in an order in which they can be executed. In many situations, one job must be completed before another job can start. For example, in software development, testing can begin only after the required code has been developed.

This project implements job scheduling using a graph data structure. Each job is represented by a vertex, and a directed edge represents a dependency between two jobs. Topological sorting is used to find a valid order of execution.

The program is implemented using Python. It uses an adjacency list to represent the graph and a queue to process jobs whose dependencies have been satisfied.

1.2 Objective:

The objective of this project is to design and implement a job scheduling system using graph data structures that arranges jobs according to their dependencies. By using graph representation and topological sorting, the system aims to generate a valid sequence of job execution while ensuring that all dependencies are satisfied.

The system aims to simplify job scheduling by identifying the correct order of dependent tasks. By using Python and a queue-based approach, the project demonstrates how graph algorithms can be applied to task management and how cycles in a graph can be detected when a valid schedule is not possible.

LITERATURE SURVEY

Graph data structures and topological sorting are important concepts in computer science. They are used to represent dependencies between tasks and determine a valid order of execution. This project uses these concepts to implement job scheduling using Python.

Year	Paper Name	Author	Conclusion
1962	"Topological Sorting of Large Networks"	Arthur B. Kahn	Presents a method for arranging tasks in an order that satisfies their dependencies.
2009	"Introduction to Algorithms, 3rd Edition"	Cormen Leiserson, Rivest and Stein	Explains graph algorithms, including topological sorting used to arrange dependent jobs.
2013	"Data Structures and Algorithms in Python"	Goodrich, Tamassia and Goldwasser	explains data structures and algorithms that can be implemented using Python.

UML DIAGRAM

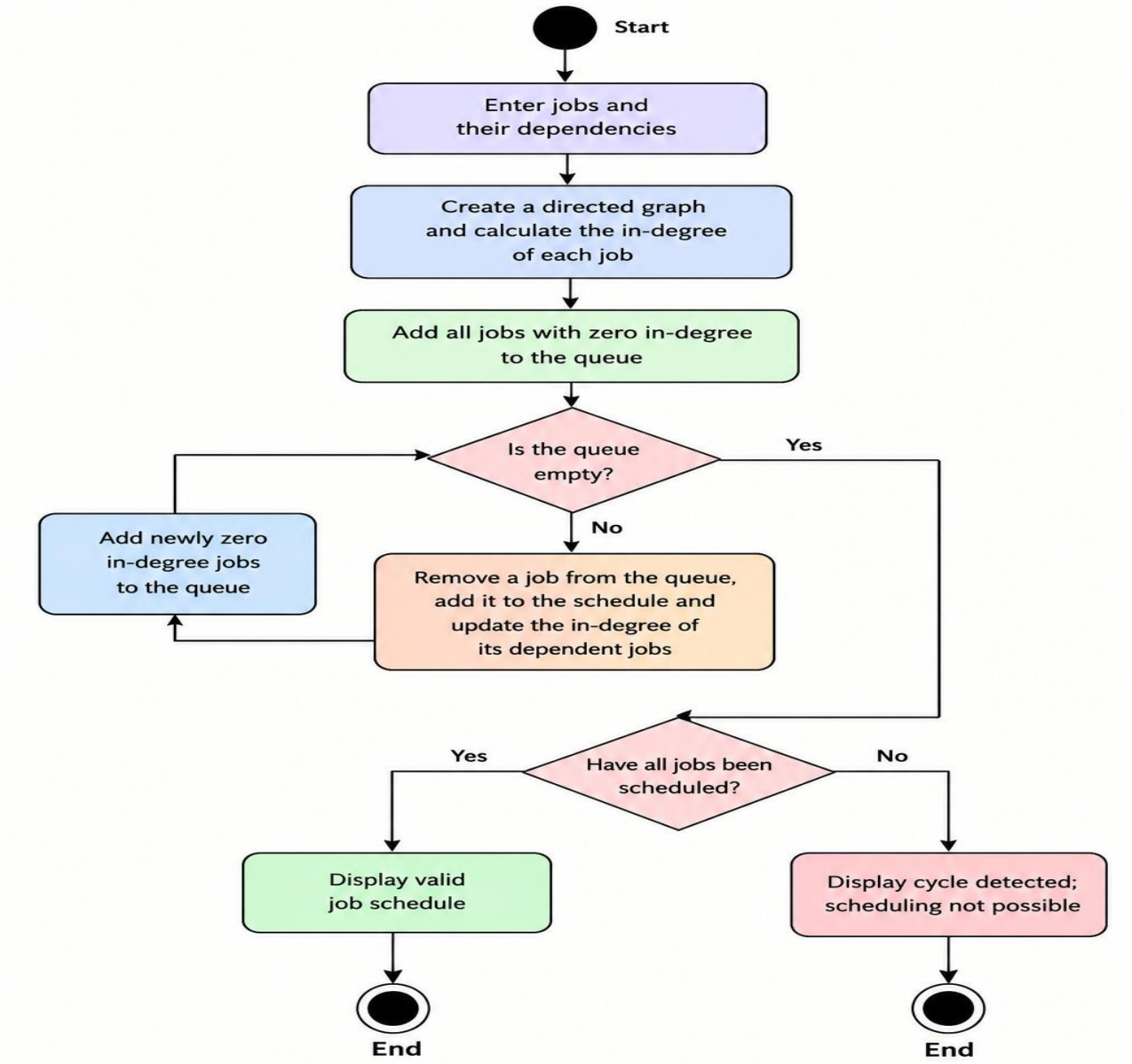


Fig. 1: UML Activity Diagram of Job Scheduling Using Graphs

SYSTEM REQUIREMENT

• HARDWARE REQUIRMENTS

The hardware requirements for implementing the Job Scheduling System are as follows:

1. **Processor:** Intel Core i3 or equivalent processor.
2. **RAM:** Minimum 4 GB RAM.
3. **Hard Disk:** Minimum 500 MB of free storage space.
4. **Input Device:** Keyboard and mouse.
5. **Output Device:** Monitor or display screen.

• SOFTWARE REQUIRMENTS

The software requirements for implementing the Job Scheduling System are as follows:

1. **Operating System:** Windows 10/11, Linux, or macOS.
2. **Programming Language:** Python 3.x.
3. **Code Editor:** Visual Studio Code, IDLE, or any Python-supported editor.
4. **Libraries:** Python's built-in collections module, which provides the deque data structure.
5. **Execution Environment:** Python interpreter to execute the program and display the output.

SYSTEM DESIGN

Existing System:

The existing job scheduling systems use various algorithms and techniques to arrange jobs according to their dependencies and execution requirements. These systems are implemented across diverse applications, including operating systems, project management tools, software development, and task automation.

Key components of existing job scheduling systems typically include:

✓ Manual Job Scheduling
✓ Priority-Based Scheduling
✓ Dependency-Based Scheduling
✓ Operating System Scheduling Algorithms

Proposed System:

The proposed job scheduling system uses graph data structures and topological sorting algorithms to arrange jobs according to their dependencies. The system is implemented using Python and uses a queue-based approach to determine a valid execution order. It can be applied to project management, software development, and task automation.

Key components of the proposed job scheduling system typically include:

✓ Directed Graph Representation
✓ Adjacency List
✓ Queue-Based Topological Sorting
✓ Cycle Detection

Directed Graph Representation:

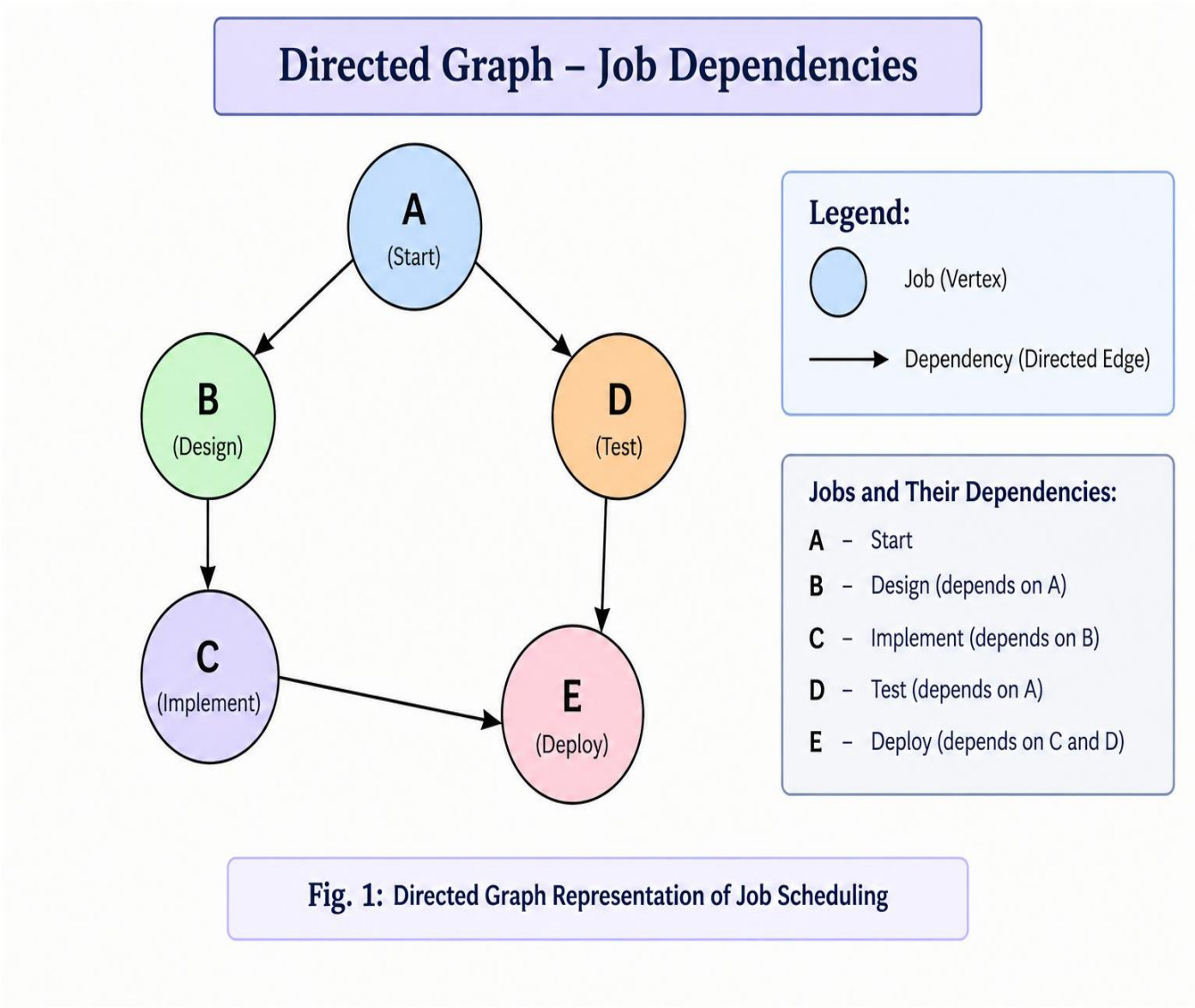
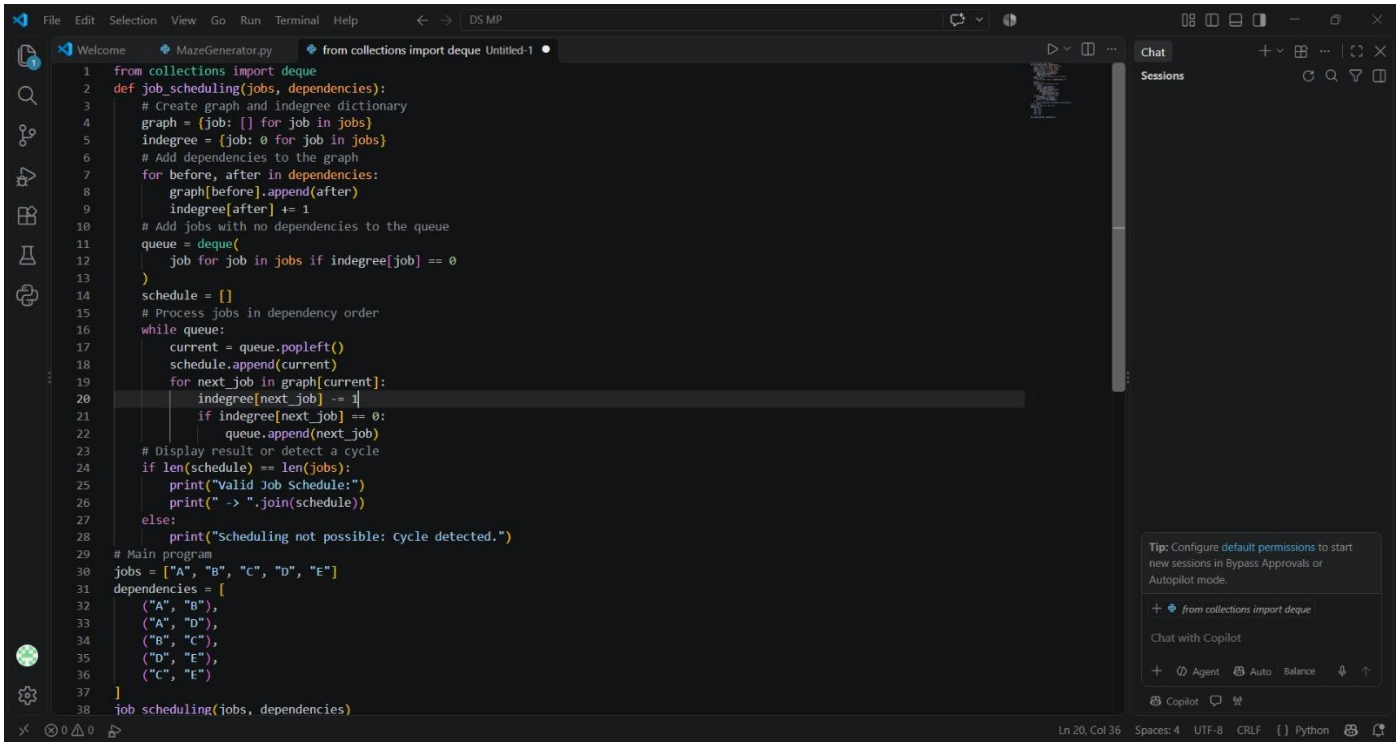


Fig. 1: Directed Graph Representation of Job Scheduling

SYSTEM CODE



```python
from collections import deque
def job_scheduling(jobs, dependencies):
    # Create graph and indegree dictionary
    graph = {job: [] for job in jobs}
    indegree = {job: 0 for job in jobs}
    # Add dependencies to the graph
    for before, after in dependencies:
        graph[before].append(after)
        indegree[after] += 1
    # Add jobs with no dependencies to the queue
    queue = deque(
        job for job in jobs if indegree[job] == 0
    )
    schedule = []
    # Process jobs in dependency order
    while queue:
        current = queue.popleft()
        schedule.append(current)
        for next_job in graph[current]:
            indegree[next_job] -= 1
            if indegree[next_job] == 0:
                queue.append(next_job)
    # Display result or detect a cycle
    if len(schedule) == len(jobs):
        print("Valid Job Schedule:")
        print(" -> ".join(schedule))
    else:
        print("Scheduling not possible: Cycle detected.")
# Main program
jobs = ["A", "B", "C", "D", "E"]
dependencies = [
    ("A", "B"),
    ("A", "D"),
    ("B", "C"),
    ("D", "E"),
    ("C", "E")
]
job_scheduling(jobs, dependencies)
```

RESULT

```
PS C:\Users\Arpita\OneDrive\Desktop\DS MP> cd "c:\Users\Arpita\OneDrive\Desktop\DS MP"; & 'c:\Users\Arpita\AppData\Roaming\Luv\python\python-3.15.0-windows-x86_64-none\python.exe' 'c:\Users\Arpita\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58037' '-' 'c:\Users\Arpita\OneDrive\Desktop\DS MP\WP1.py'
Valid Job Schedule:
A -> B -> D -> C -> E
PS C:\Users\Arpita\OneDrive\Desktop\DS MP>
```

APPLICATIONS

The applications of a Job Scheduling System using Graphs demonstrate the usefulness of graph data structures and topological sorting in arranging tasks according to their dependencies. This system can be applied across various fields of computer science and task management, helping ensure that jobs are executed in the correct order.

Project management tools can utilize graph-based job scheduling to arrange project activities according to their dependencies. By identifying which tasks must be completed first, these systems can help project managers plan work efficiently, avoid scheduling conflicts, and ensure the timely completion of projects.

Software development and build systems can use job scheduling to determine the correct order for compiling source files and building software modules. When one module depends on another, the system ensures that the required module is processed first, reducing dependency-related errors during the build process.

Operating systems and task automation applications can also benefit from graph-based scheduling. Tasks that depend on the completion of other tasks can be arranged in a valid sequence, helping automate workflows and coordinate multiple operations efficiently.

Educational institutions and organizations can use dependency-based scheduling to plan activities such as assignments, examinations, and training programs. By representing dependent activities as a directed graph, the system can help identify a suitable order for completing tasks and detect cycles that make a valid schedule impossible.

.

FUTURE WORK

The future of the Job Scheduling Using Graphs project may involve experimenting with advanced scheduling algorithms and techniques to improve the efficiency of job execution. Additional features such as priority-based scheduling, execution time estimation, and resource management can be explored to enhance the performance of the existing system.

By exploring these avenues for future work, the system can be extended to handle larger numbers of jobs, support parallel execution of independent tasks, and provide a graphical user interface for easier interaction. These improvements can increase the efficiency, scalability, and usability of the job scheduling system, making it more useful for real-world applications.

CONCLUSION

In this project, we developed a Job Scheduling System using graph data structures and Python. The goal was to determine a valid execution order for jobs based on their dependencies.

We represented jobs as vertices and their dependencies as directed edges in a graph. The system uses an adjacency list to store the graph and Kahn's topological sorting algorithm to arrange jobs in the correct sequence. A queue is used to process jobs whose dependencies have been satisfied, while cycle detection identifies situations where a valid schedule cannot be generated.

This project demonstrates the practical application of graph data structures, queue operations, and topological sorting in solving job scheduling problems. It also provides an understanding of how graph algorithms can be used to organize dependent tasks efficiently.

REFERENCE

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*.

[2] Sedgewick, R., & Wayne, K. (2011). *Algorithms*.

[3] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*.

[4] Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558–562.

[5] Python Software Foundation. *Python 3 Documentation: collections — Container datatypes*.